

```

for (j=0;j<=n-2;j++)
    for (k=n-2;k>=j;k--)
        d[k] -= cnst*d[k+1];
}

```

We accomplish the shift by synthetic division, a miracle of high-school algebra.

CITED REFERENCES AND FURTHER READING:

Acton, F.S. 1970, *Numerical Methods That Work*, 1990, corrected edition (Washington, DC: Mathematical Association of America), pp. 59, 182–183 [synthetic division].

5.11 Economization of Power Series

One particular application of Chebyshev methods, the *economization of power series*, is an occasionally useful technique, with a flavor of getting something for nothing.

Suppose that you know how to compute a function by the use of a convergent power series, for example,

$$f(x) \equiv \frac{1}{2} - \frac{x}{4} + \frac{x^2}{8} - \frac{x^3}{16} + \cdots \quad (5.11.1)$$

(This function is actually just $1/(x+2)$, but pretend you don't know that.) You might be doing a problem that requires evaluating the series many times in some particular interval, say $[0, 1]$. Everything is fine, except that the series requires a large number of terms before its error (approximated by the first neglected term, say) is tolerable. In our example, with $x = 1$, it takes about 30 terms before the first neglected term is $< 10^{-9}$.

Notice that because of the large exponent in x^{30} , the error is *much smaller* than 10^{-9} everywhere in the interval except at the very largest values of x . This is the feature that allows “economization”: If we are willing to let the error elsewhere in the interval rise to about the same value that the first neglected term has at the extreme end of the interval, then we can replace the 30-term series by one that is significantly shorter.

Here are the steps for doing this:

1. Compute enough coefficients of the power series to get accurate function values everywhere in the range of interest.
2. Change variables from x to y , as in equation (5.8.10), to map the x interval into $-1 \leq y \leq 1$.
3. Find the Chebyshev series (like equation 5.8.8) that exactly equals your truncated power series.
4. Truncate this Chebyshev series to a smaller number of terms, using the coefficient of the first neglected Chebyshev polynomial as an estimate of the error.
5. Convert back to a polynomial in y .
6. Change variables back to x .

We already have tools for all of the steps, except for steps 2 and 3. Step 2 is exactly the inverse of the routine `pcshft` (§5.10), which mapped a polynomial from y (in the interval $[-1, 1]$) to x (in the interval $[a, b]$). But since equation (5.8.10) is a linear relation between x and y , one can also use `pcshft` for the inverse. The inverse of

$$\text{pcshft}(a, b, d, n)$$

turns out to be (you can check this)

```

void ipcshft(Doub a, Doub b, VecDoub_IO &d) {
    pcshft(-(2.+b+a)/(b-a), (2.-b-a)/(b-a), d);
}

```

`pcshft.h`

Step 3 requires a new Chebyshev constructor, one that computes Chebyshev coefficients from a vector of polynomial coefficients. The following code accomplishes this. The algorithm is based on constructing the polynomial by the technique of §5.3 starting with the highest coefficient $d[n-1]$ and using the recurrence of equation (5.8.2) written in the form

$$\begin{aligned} xT_0 &= T_1 \\ xT_n &= \frac{1}{2}(T_{n+1} + T_{n-1}), \quad n \geq 1. \end{aligned} \quad (5.11.2)$$

The only subtlety is to multiply the coefficient of T_0 by 2 since it gets used with a factor 1/2 in equation (5.8.8).

chebyshev.h

```
Chebyshev::Chebyshev(VecDoub &d)
: n(d.size()), m(n), c(n), a(-1.), b(1.)
Inverse of routine polycofs in Chebyshev: Given an array of polynomial coefficients d[0..n-1],
construct an equivalent Chebyshev object.
{
    c[n-1]=d[n-1];
    c[n-2]=2.0*d[n-2];
    for (Int j=n-3;j>=0;j--) {
        c[j]=2.0*d[j]+c[j+2];
        for (Int i=j+1;i<n-2;i++) {
            c[i] = (c[i]+c[i+2])/2;
        }
        c[n-2] /= 2;
        c[n-1] /= 2;
    }
}
```

Putting them all together, steps 2 through 6 will look something like this (starting with a vector `powser` of power series coefficients):

```
ipcshft(a,b,powser);
Chebyshev cpowser(powser);
cpowser.setm(1.e-9);
VecDoub d=cpowser.polycofs();
pcshft(a,b,d);
```

In our example, by the way, the number of terms required for 10^{-9} accuracy is reduced from 30 to 9. Replacing a 30-term polynomial with a 9-term polynomial without any loss of accuracy — that does seem to be getting something for nothing. Is there some magic in this technique? Not really. The 30-term polynomial defined a function $f(x)$. Equivalent to economizing the series, we could instead have evaluated $f(x)$ at enough points to construct its Chebyshev approximation in the interval of interest, by the methods of §5.8. We would have obtained just the same lower-order polynomial. The principal lesson is that the rate of convergence of Chebyshev coefficients has nothing to do with the rate of convergence of power series coefficients; and it is the *former* that dictates the number of terms needed in a polynomial approximation. A function might have a *divergent* power series in some region of interest, but if the function itself is well-behaved, it will have perfectly good polynomial approximations. These can be found by the methods of §5.8, but *not* by economization of series. There is slightly less to economization of series than meets the eye.

CITED REFERENCES AND FURTHER READING:

Acton, F.S. 1970, *Numerical Methods That Work*, 1990, corrected edition (Washington, DC: Mathematical Association of America), Chapter 12.